Biological Algorithms - Sequence alignments

Computational Methods for Bioinformatics 2024/2025 - Manfredi Matteo

Summary

1. String Metrics and Alignments
2. Needleman–Wunsch Algorithm
3. Smith–Waterman Algorithm

String Metrics and Alignments

string metrix, distance or similarity between two strings

String Metric

- Metric that measures a **distance** (or a **similarity**) between two **text** strings

- Application in different fields
  - Linguistic
  - Information Theory
  - Natural Language Processing
  - **Bioinformatics**

Summary

Edit distance

edit distance= metrix that measure how many operations (and how is the cost) to one string to another

- The most common group of string metrics are called edit distances
- **Cost of operations needed to transform one string into the other**
- Possible operations:
  - Substitution
  - Insertion / Deletion
  - Transposition
- Different costs

```
CAT
| |
CUT
```

1 substitution: A to U, changing 1 character

```
MONKEY
||| ||
MON-EY
```

when you insert or delete means remove than 1

```
FORM
|  |
FROM
```

FORM to FROM transpose the O and R, consider 1 operation; not usually use in biological

Summary

Hamming Distance

easiest distance that you can compute, the hamming is an edit distance that consider only substitutions as possible operations; this also means you can only substitute 1 charcater to another -->distance with the same number of characters

- Edit distance that considers only **substitutions** as possible operations
- Applicable only to strings with **same length**
- Example:

```
ACCCTCGCTAGATAAATAGATCTGATAG
||x|||||||||x|||||x||||x|||
ACTCTCGCTAGATGAATAGGTCTGTTAG
```

Hamming Distance = 4

the distance is the cost of the operation that you perform, each substitutions have a cost, and in this case you just count

Summary

Distance or Similarity metric?

- Distance:

  this are two related concept, but not the same: distance usually compute a score that is positive

  - High positive score ➔ Distant strings
  - Score of 0 ➔ Identical strings

- Similarity:

  when you use biological sequences: usually compare sequences that you know that are similar, you know to compute a score (high when the sequences are similar..)

  - High positive score ➔ Similar strings
  - High negative score ➔ Distant strings

- In bioinformatic applications we usually care about **similarity**

Summary

Hamming Distance

could define something similar: hamming similarity, score that high possible when the two sequences are similar.. but what do you change? instead of counting is positive way; we want to way positevely the characters are the same. when you increase for every part of the strings that have the same...not only decrease

- Edit distance that considers only **substitutions** as possible operations
- Applicable only to strings with **same length**
- Example:

```
ACCCTCGCTAGATAAATAGATCTGATAG
||x||||||||||x|||||x||||x|||
ACTCTCGCTAGATGAATAGGTCTGTTAG
```

Hamming Distance = 4

Similarity = ?

Summary

Hamming Distance

- Edit distance that considers only **substitutions** as possible operations
- Applicable only to strings with **same length**
- Example:

```
ACCCTCGCTAGATAAATAGATCTGATAG
||x|||||||||||x|||||x||||x|||
ACTCTCGCTAGATGAATAGGTCTGTTAG
```

Hamming Distance = 4

Similarity = + 24

Summary

Hamming Distance

- Edit distance that considers only **substitutions** as possible operations
- Applicable only to strings with **same length**
- Example:

```
ACCCTCGCTAGATAAATAGATCTGATAG
||x|||||||||||x|||||x||||x|||
ACTCTCGCTAGATGAATAGGTCTGTTAG
```

Hamming Distance = 4

Similarity = + 24 - 4 = 20

Summary

Hamming Distance - Implementation

Define a class **AlnSeq**

new class: AlnSeq, when you define a class you create a new data type that usually have some meaning of object

- Attributes:
  - First string
  - Second string

two attribute= each object of that class will have a set of internal variables

first : first sequence and the second contain the second sequence

- Methods:
  - Compute Hamming distance
  - Compute Hamming similarity

method: is a function define inside the class, and it is a function that belong of the object of this class
2 methos for stance: 1 the hamming distance and other to compute an hamming similarity

when you define a function you use def.. when the class are the same

Summary

naming convention: something is good practice to follow

Biologically Relevant Scores

- When considering similarity in biology, not every substitution is the same
  - Some nucleic/amino acids are interchangeable ➔ Small positive score
  - Some nucleic/amino acids are very different ➔ Negative score

- Different substitution matrices can be adopted
  - Transition Transversion Matrix for DNA
  - BLOSUM and PAM for Proteins

Summary

Transition Transversion Matrix

- Useful for measuring the similarity of two DNA sequences
- Transversion: **A ↔ T** **A ↔ C** **G ↔ T** **G ↔ C**
- Transition: **A ↔ G** **T ↔ C**

```
AAAA
|XX/
ATCG
```

Score(A,A) + Score(A,T) + Score(A,C) + Score(A,G) =

= 2 + (-1) + (-1) + 0 = 0

	A	T	C	G
A	2	-1	-1	0
T	-1	2	0	-1
C	-1	0	2	-1
G	0	-1	-1	2

Summary

Point Accepted Mutation Matrices

- Useful for measuring the similarity of two Protein sequences

- **PAM_n[i,j]** → Likelihood of res **i** being replaced with res **j** through a series of one or more PAMs during a time frame long enough for **n** mutation events to occur per 100 residues

- Most adopted → **PAM250**

Summary

Point Accepted Mutation Matrices

	A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V
A	2	-2	0	0	-2	0	0	1	-1	-1	-2	-1	-1	-4	1	1	1	-6	-3	0
R	-2	6	0	-1	-4	1	-1	-3	2	-2	-3	3	0	-4	0	0	-1	2	-4	-2
N	0	0	2	2	-4	1	1	0	2	-2	-3	1	-2	-4	-1	1	0	-4	-2	-2
D	0	-1	2	4	-5	2	3	1	1	-2	-4	0	-3	-6	-1	0	0	-7	-4	-2
C	-2	-4	-4	-5	12	-5	-5	-3	-3	-2	-6	-5	-5	-4	-3	0	-2	-8	0	-2
Q	0	1	1	2	-5	4	2	-1	3	-2	-2	1	-1	-5	0	-1	-1	-5	-4	-2
E	0	-1	1	3	-5	2	4	0	1	-2	-3	0	-2	-5	-1	0	0	-7	-4	-2
G	1	-3	0	1	-3	-1	0	5	-2	-3	-4	-2	-3	-5	-1	1	0	-7	-5	-1
H	-1	2	2	1	-3	3	1	-2	6	-2	-2	0	-2	-2	0	-1	-1	-3	0	-2
I	-1	-2	-2	-2	-2	-2	-2	-3	-2	5	2	-2	2	1	-2	-1	0	-5	-1	4
L	-2	-3	-3	-4	-6	-2	-3	-4	-2	2	6	-3	4	2	-3	-3	-2	-2	-1	2
K	-1	3	1	0	-5	1	0	-2	0	-2	-3	5	0	-5	-1	0	0	-3	-4	-2
M	-1	0	-2	-3	-5	-1	-2	-3	-2	2	4	0	6	0	-2	-2	-1	-4	-2	2
F	-4	-4	-4	-6	-4	-5	-5	-5	-2	1	2	-5	0	9	-5	-3	-3	0	7	-1
P	1	0	-1	-1	-3	0	-1	-1	0	-2	-3	-1	-2	-5	6	1	0	-6	-5	-1
S	1	0	1	0	0	-1	0	1	-1	-1	-3	0	-2	-3	1	2	1	-2	-3	-1
T	1	-1	0	0	-2	-1	0	0	-1	0	-2	0	-1	-3	0	1	3	-5	-3	0
W	-6	2	-4	-7	-8	-5	-7	-7	-3	-5	-2	-3	-4	0	-6	-2	-5	17	0	-6
Y	-3	-4	-2	-4	0	-4	-4	-5	0	-1	-1	-4	-2	7	-5	-3	-3	0	10	-2
V	0	-2	-2	-2	-2	-2	-2	-1	-2	4	2	-2	2	-1	-1	-1	0	-6	-2	4

Summary

BLOcks SUbstitution Matrices

- Useful for measuring the similarity of two Protein sequences

- **$BLOSUM_n[i,j]$ ➔** Log odd ratio of the frequency of occurrence of residues i and j in the Multiple Sequence Alignment of sequences clustered at n% of sequence identity, over the probability that they might align by chance

- Most adopted ➔ **BLOSUM62**

Summary

BLOcks SUbstitution Matrices

	A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V
A	4	-1	-2	-2	0	-1	-1	0	-2	-1	-1	-1	-1	-2	-1	1	0	-3	-2	0
R	-1	5	0	-2	-3	1	0	-2	0	-3	-2	2	-1	-3	-2	-1	-1	-3	-2	-3
N	-2	0	6	1	-3	0	0	0	1	-3	-3	0	-2	-3	-2	1	0	-4	-2	-3
D	-2	-2	1	6	-3	0	2	-1	-1	-3	-4	-1	-3	-3	-1	0	-1	-4	-3	-3
C	0	-3	-3	-3	9	-3	-4	-3	-3	-1	-1	-3	-1	-2	-3	-1	-1	-2	-2	-1
Q	-1	1	0	0	-3	5	2	-2	0	-3	-2	1	0	-3	-1	0	-1	-2	-1	-2
E	-1	0	0	2	-4	2	5	-2	0	-3	-3	1	-2	-3	-1	0	-1	-3	-2	-2
G	0	-2	0	-1	-3	-2	-2	6	-2	-4	-4	-2	-3	-3	-2	0	-2	-2	-3	-3
H	-2	0	1	-1	-3	0	0	-2	8	-3	-3	-1	-2	-1	-2	-1	-2	-2	2	-3
I	-1	-3	-3	-3	-1	-3	-3	-4	-3	4	2	-3	1	0	-3	-2	-1	-3	-1	3
L	-1	-2	-3	-4	-1	-2	-3	-4	-3	2	4	-2	2	0	-3	-2	-1	-2	-1	1
K	-1	2	0	-1	-3	1	1	-2	-1	-3	-2	5	-1	-3	-1	0	-1	-3	-2	-2
M	-1	-1	-2	-3	-1	0	-2	-3	-2	1	2	-1	5	0	-2	-1	-1	-1	-1	1
F	-2	-3	-3	-3	-2	-3	-3	-3	-1	0	0	-3	0	6	-4	-2	-2	1	3	-1
P	-1	-2	-2	-1	-3	-1	-1	-2	-2	-3	-3	-1	-2	-4	7	-1	-1	-4	-3	-2
S	1	-1	1	0	-1	0	0	0	-1	-2	-2	0	-1	-2	-1	4	1	-3	-2	-2
T	0	-1	0	-1	-1	-1	-1	-2	-2	-1	-1	-1	-1	-2	-1	1	5	-2	-2	0
W	-3	-3	-4	-4	-2	-2	-3	-2	-2	-3	-2	-3	-1	1	-4	-3	-2	11	2	-3
Y	-2	-2	-2	-3	-2	-1	-2	-3	2	-1	-1	-2	-1	3	-3	-2	-2	2	7	-1
V	0	-3	-3	-3	-1	-2	-2	-3	-3	3	1	-2	1	-1	-2	-2	0	-3	-1	4

Summary

Reading Matrices - Implementation

Define a class **SubstitutionMatrix**

- Attributes:
  - Data structure to hold the matrix values
- Methods:
  - Implement custom **`__getitem__`** method to index the object with the **`[]`** operator

Extend the class **AlnSeq**

- New methods:
  - Compute Hamming similarity based on a given substitution matrix

Summary

Levenshtein Distance

- Edit distance that considers **substitutions** and **indels** as possible operations

- Example:

```
KITTEN    →  KITTEN-
             X|||X|^
SITTING   →  SITTING
```

Levenshtein Distance = 3

Summary

Levenshtein vs Hamming

- For sequences of same length → Hamming Distance >= Levenshtein Distance
- Example:

```
FLAW   →   FLAW-
XXXX   →   v|||^
LAWN   →   -LAWN
```

Hamming Distance = 4

Levenshtein Distance = 2

- **We need to align the two sequences**

Summary

Sequence Alignment

- Each character of a sequence is matched with a character of the other sequence, or with a gap

- After the alignment we have two (gapped) strings with the same length

- We can compute the Hamming distance by defining the cost of matching a character with a gap

Summary

Optimal Alignment

- How can we find the alignment that minimizes the distance (or maximizes the similarity score)?

- First idea → **Brute Force**
  - We compute the score for each possible alignment to find the optimal one

- How many possibilities?

Summary

Simple case - No internal gaps

- If we don’t allow internal gaps, we can slide the two sequences

```
--TCA    -TCA    TCA     TCA     TCA-    TCA--
^^vvv    ^Xvv    |Xv     vX|     vvXv    vvv^^
TA---    TA--    TA-     -TA     --TA    ---TA
```

- $N_{aln}(ls1, ls2) = ls1 + ls2 + 1$

Summary

Complex case

```
--TCA    -TCA    -T-CA    -TCA    -TC-A    -TCA    -TCA-
TA---    TA--    T-A--    T-A-    T--A-    T--A    T---A

T-CA     TCA     TC-A     TCA     TCA-
TA--     TA--    T-A--    T-A     T--A

T--CA    T-CA    T-C-A    T-CA    T-CA-
-TA--    -TA-    -T-A-    -T-A    -T--A

TC-A     TCA     TCA-     TC--A   TC-A     TC-A-    TCA-     TCA--
-TA-     -TA     -T-A     --TA-   --TA     --T-A    --TA     ---TA
```

Summary

Complex case

- We can build the alignments incrementally with branching
- First position:
  - Match
  - Insertion (gap in first sequence)
  - Deletion (gap in second sequence)
- When we used all characters of a sequence, we can only align all the ones remaining in the other with gaps

Summary

Complex case

$$N_{aln}(ls1,\ ls2) = \begin{cases} 1 & \text{if } ls1 == 0 \\ 1 & \text{if } ls2 == 0 \\ N_{aln}(ls1 - 1,\ ls2) + N_{aln}(ls1,\ ls2 - 1) + N_{aln}(ls1 - 1,\ ls2 - 1) & \text{otherwise} \end{cases}$$

$N_{aln}(3,3) = 63$

$N_{aln}(5,5) = 1'683$

$N_{aln}(9,9) = 1'462'563$

Summary

Complex case

- Not computationally feasible to brute force all possible alignments

- We need a better strategy

- Let’s try with a small example
  - Align “TCA” with “TA”
  - Cost of gap = -2
  - Transition Transversion Matrix for matches

Summary

Complex case

- The optimal alignment is the best amongst three choices
  - Open a gap in the first sequence and align “TCA” with “T”

```
[TCA]-
[T  ]A
```

  - Open a gap in the second sequence and align “TC” with “TA”

```
[TC]A
[TA]-
```

  - Align the last residues and align “TC” with “T”

```
[TC]A
[T ]A
```

- The solution can be derived by the solution to sub-problems

Summary

Dynamic Programming

- Algorithmic paradigm

- Can be applied to problems with two properties:
  - Optimal substructure → The optimal solution depends on smaller sub-problems
  - Overlapping subproblems → The solution to the sub-problems is reused several times

- Basic idea: Keep in memory the solutions to the sub-problems and iteratively find the solution to bigger problems

Summary

Levenshtein Distance - Implementation

Extend the class **AlnSeq**

- New methods:
  - Compute the Levenshtein similarity for two aligned sequences based on a given substitution matrix and on a gap penalty
  - Implement custom **`__str__`** method to print the sequences in three lines. The first shows the first sequence, the second shows a match string, and the third shows the second sequence
    - **`X`** → Substitution
    - **`|`** → Identity
    - **`^`** → Insertion
    - **`v`** → Deletion

Summary

EXTRA: Number of possible alignments - Implementation

Define a recursive function

- Parameters:
  - Length of the first sequence
  - Length of the second sequence
- Return:
  - Number of possible alignments between the two sequences

Summary

Needleman–Wunsch Algorithm

Initialization

	-	T	C	A
-				
T				
A				

Summary

Initialization

	-	T	C	A
-	0			
T				
A				

Summary

Initialization

	-	T	C	A
-	0	gap * 1	gap * 2	gap * 3
T	gap * 1			
A	gap * 2			

Summary

Initialization

	-	T	C	A
-	0	-2	-4	-6
T	-2			
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	max(-2+gap, 0+match(T,T), -2+gap)		
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	max(-4, 2, -4)		
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2		
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	max(-4+gap, -2+match(C,T), 2+gap)	
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	max(-6, -2, 0)	
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	max(-6+gap, -4+match(A,T), 0+gap)
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	max(-8, -5, -2)
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4			

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	max(2+gap, -2+match(T,A), -4+gap)		

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	max(0, -3, -6)		

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0		

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	max(0+gap, 2+match(C,A), 0+gap)	

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	max(-2, 1, -2)	

Summary

Solving subproblems

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	1	

Summary

Final Solution

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	1	max(-2+gap, 0+match(A,A), 1+gap)

Summary

Final Solution

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	1	max(-4, 2, -1)

Summary

Final Solution

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	1	2

Summary

Reconstructing the alignment

- Up to this point we computed the score of the best alignment

- What does the actual alignment look like?

- We need to keep track of the choices we make while filling the matrix

Summary

Initialization

	-	T	C	A
-	0	-2	-4	-6
T	-2			
A	-4			

Summary

Filling the Matrix

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	1	2

Summary

Backtracking

A
A

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	1	2

Summary

Backtracking

CA
-A

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	1	2

Summary

Backtracking

TCA
T-A

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	1	2

Summary

Backtracking

Final Alignment: TCA
Score = 2 T-A

	-	T	C	A
-	0	-2	-4	-6
T	-2	2	0	-2
A	-4	0	1	2

Summary

Needleman and Wunsch - Implementation

Define a class **SeqPair**

- Attributes:
  - First string
  - Second string
- Methods:
  - Compute the matrices for the global alignment
  - Reconstruct the global alignment from the matrices

Summary

Smith–Waterman Algorithm

Global vs Local alignment

- The Needleman-Wunsch Algorithm finds the best global alignment
- A local alignment is an alignment between substrings of the target sequences
- The Smith-Waterman Algorithm makes small adjustments to the NW to find the optimal local alignments
    - No starting and ending gaps
    - Discard parts of the alignment that would contribute with a negative score

Summary

Initialization

	-	T	G	A
-	0	0	0	0
G	0			
A	0			

Summary

Filling the Matrix

	-	T	G	A
-	0	0	0	0
G	0	max(0+gap, 0+match(T,G), 0+gap, 0)		
A	0			

Summary

Filling the Matrix

	-	T	G	A
-	0	0	0	0
G	0	max(-2,-1,-2,0)		
A	0			

Summary

Filling the Matrix

	-	T	G	A
-	0	0	0	0
G	0	0		
A	0			

Summary

Filling the Matrix

	-	T	G	A
-	0	0	0	0
G	0	0	2	0
A	0	0	0	4

Summary

Backtracking

A
A

	-	T	G	A
-	0	0	0	0
G	0	0	2	0
A	0	0	0	4

Summary

Backtracking

GA
GA

	-	T	G	A
-	0	0	0	0
G	0	0	2	0
A	0	0	0	4

Summary

Backtracking

Final Alignment: GA
Score = 4 GA

	-	T	G	A
-	0	0	0	0
G	0	0	2	0
A	0	0	0	4

Summary

Smith and Waterman - Implementation

Extend the class **SeqPair**

- New methods:
  - Compute the matrices for the local alignment
  - Reconstruct the local alignment from the matrices

Summary